

# European Soccer Database

Final Project · Deliverable 4 · soccer\_proj

Emmanuel Arthur · Database Class · Spring 2026

# Dataset overview

Source: Kaggle European Soccer Database (SQLite export → CSV → PostgreSQL)

Scope: 11 countries, 11 leagues, ~26K matches (2008–2016), teams, players, attributes

Business context: same kind of data clubs, sports media, and analytics vendors use for reporting

Schema soccer\_proj with 7 base tables + views for common joins

Total loaded rows across all tables: 222,796

# Why I chose this dataset

I have followed European club soccer since childhood (Man United, then Bayern Munich)

The domain is familiar: leagues, seasons, home/away, points, streaks, betting odds

Business-style questions: league scoring trends, defensive strength, player development, odds vs results

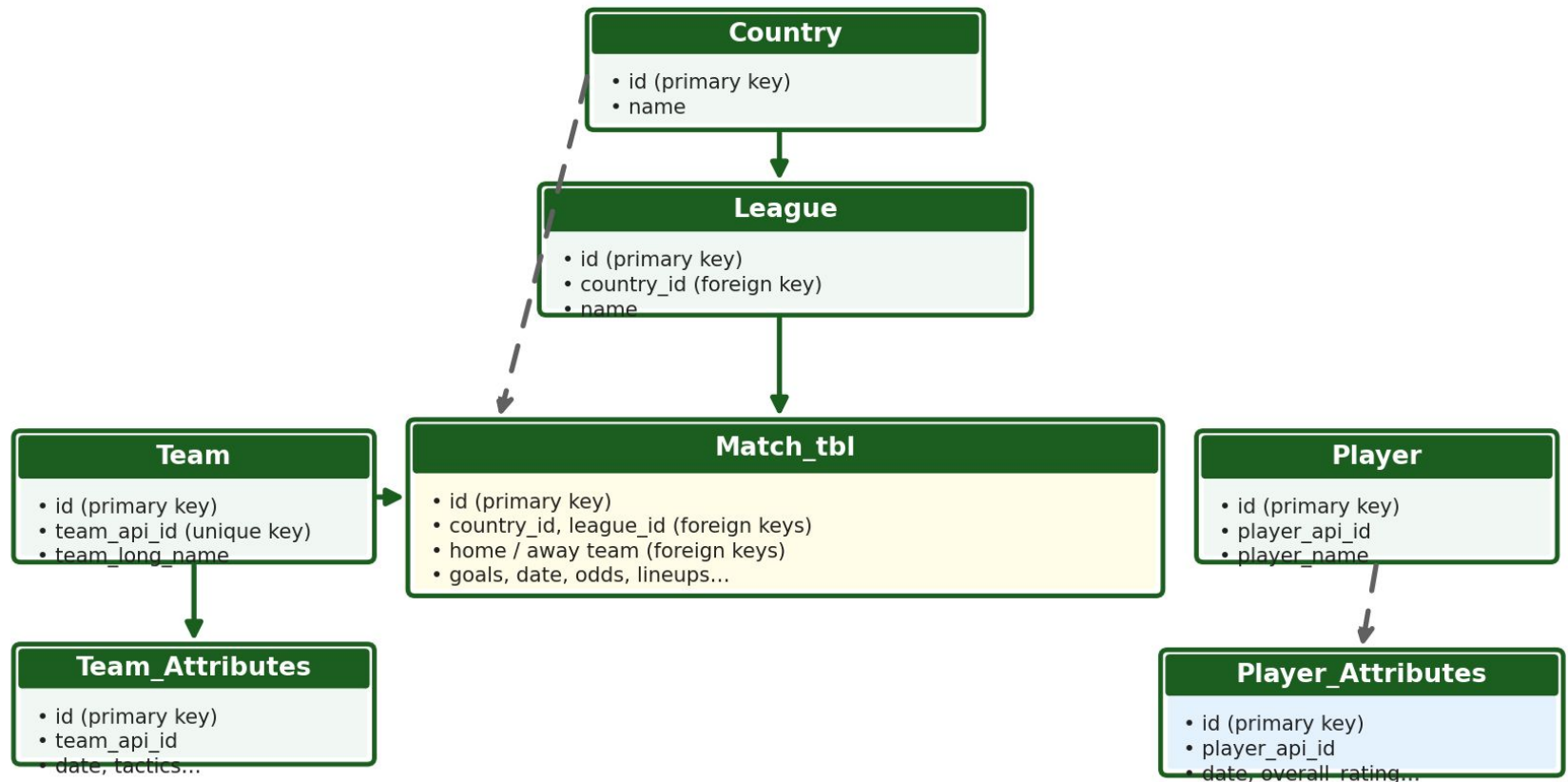
Good mix of dimensions (country, league, team, player) and facts (matches, ratings)

Graduate requirement: 30 English questions that map to real SELECT workloads

# ER diagram — soccer\_proj

Initial and final use the same 7 tables

## soccer\_proj — entity-relationship diagram (7 base tables)



Solid arrow = enforced foreign key in PostgreSQL

Dashed arrow = logical link in source data (may not be enforced)

Data model for business reporting: Country/League/Team/Player = who & where; Match\_tbl = transactions (each game); attributes = snapshots over time. Views = ready-made reports.

# Hardest part

Loading and typing: SQLite → CSV → Postgres with consistent column names and NULL handling

Match\_tbl is very wide (lineup API IDs, many bookmaker columns) — easy to get joins wrong

Writing correct multi-step SQL (common table expressions, window functions) for streaks and lineup-based ratings

Deciding which foreign keys to enforce vs document-only when source data has orphan keys

Keeping 30 project questions aligned with what the data actually supports

# What I learned

Design first: ERD and keys matter before writing dozens of queries

JOIN choice changes results (INNER vs LEFT) — always check row counts

Views encapsulate repeated 4–5 table joins; indexes help filter/join columns on Match\_tbl

Window functions (partition rows per team/season, compare to prior rows) for streaks and trends

Documentation + reproducible load scripts save time at deliverable deadline

# SQL confidence now

Comfortable with the most common SQL: SELECT, JOINS, GROUP BY, HAVING, subqueries, CASE

Comfortable creating views (for example league\_match\_summary\_view, match\_context\_view)

Overall: much more confident than at the start of the semester — I can debug by counting rows

DBeaver plus the EXPLAIN command helped me see when a query execution plan looked expensive

# What I would do next (more time)

Enforce more foreign keys after data cleaning; add CHECK constraints on goals and dates

Materialized views or summary tables for league/season dashboards

More indexes tuned using EXPLAIN on the slowest of the 30 queries

Data quality report: missing lineups, duplicate player identifiers, odds outliers

Optional: BI dashboard (Tableau/Power BI) on views for non-technical stakeholders



# DEMO: Tables + row counts

DBeaver · schema soccer\_proj

Demo these core tables (foreign key relationships) in DBeaver — columns + row count:

Country — 11 rows (parent; League.country\_id → Country.id)

League — 11 rows (country\_id foreign key; Match\_tbl.league\_id → League.id)

Team — 299 rows (home/away team\_api\_id on Match\_tbl)

Match\_tbl — 25,979 rows (hub: country\_id, league\_id, home/away teams, goals)

All 7 tables combined — 222,796 rows total

# DEMO: Intermediate query (Q16)

Joins + GROUP BY + AVG (no CASE)

Question: Which countries had the highest average goals per match?

Tables: Match\_tbl → League → Country · INNER JOIN, GROUP BY, AVG (no CASE)

**SQL (copy into DBeaver on soccer\_proj):**

```
SELECT c.name AS country,  
       AVG((m.home_team_goal + m.away_team_goal)::numeric) AS avg_goals_per_match  
FROM   soccer_proj."Match_tbl" m  
JOIN   soccer_proj."League" l ON l.id = m.league_id  
JOIN   soccer_proj."Country" c ON c.id = l.country_id  
GROUP BY c.name  
ORDER BY avg_goals_per_match DESC;
```

# DEMO Q16 — Query results

DBeaver: demo1 - Netherlands leads avg goals per match

The screenshot shows the DBeaver SQL Editor interface. The left sidebar displays the database structure for 'postgres' on 'localhost:5432', including schemas 'public' and 'spy', and various tables like 'affiliation', 'agent', 'language', etc. The main editor window shows a SQL query titled 'demo1' with the following text:

```
-- Which countries had the highest average goals per match across all their leagues?
SELECT c.name AS country,
       AVG((m.home_team_goal + m.away_team_goal)::numeric) AS avg_goals_per_match
FROM soccer_proj."Match_tbl" m
JOIN soccer_proj."League" l ON l.id = m.league_id
JOIN soccer_proj."Country" c ON c.id = l.country_id
GROUP BY c.name
ORDER BY avg_goals_per_match DESC;
```

Below the query editor, the results are displayed in a table titled 'Country 1'. The table has two columns: 'country' and 'avg\_goals\_per\_match'. The results are as follows:

| country     | avg_goals_per_match |
|-------------|---------------------|
| Netherlands | 3.0808823529        |
| Switzerland | 2.929676512         |
| Germany     | 2.9015522876        |
| Belgium     | 2.8015046296        |
| Spain       | 2.7671052632        |
| England     | 2.7105263158        |
| Scotland    | 2.6337719298        |

At the bottom of the results pane, it indicates '11 rows fetched' and '0.126s (0.001s fetch)' on '2026-05-25 at 12:52:58'. A tooltip is visible over the 'Netherlands' row, showing 'ODIN ID: emmanart' and 'Name: Emmanuel Arthur'.

# DEMO: Advanced query (Q18)

Join with OR + conditional aggregates

Question: Which teams most frequently kept clean sheets?

Tables: Team + Match\_tbl · JOIN with OR · two CASE inside SUM · GROUP BY

**SQL (copy into DBeaver on soccer\_proj):**

```
SELECT t.team_long_name,  
       SUM(CASE WHEN m.home_team_api_id = t.team_api_id AND m.away_team_goal = 0 THEN 1 ELSE 0 END  
           + CASE WHEN m.away_team_api_id = t.team_api_id AND m.home_team_goal = 0 THEN 1 ELSE 0 END) AS clean_sheets  
FROM soccer_proj."Team" t  
JOIN soccer_proj."Match_tbl" m  
  ON m.home_team_api_id = t.team_api_id OR m.away_team_api_id = t.team_api_id  
GROUP BY t.team_long_name  
ORDER BY clean_sheets DESC  
LIMIT 30;
```

# DEMO Q18 — Query results

DBeaver: demo2 - Celtic leads clean sheets

The screenshot shows the DBeaver SQL Editor interface. The left sidebar displays the database structure for a PostgreSQL instance, including schemas (public, spy) and tables (affiliation, agent, language, mission, newagent, securityclearance, skill, skillrel, team, teamrel). The main editor window contains a SQL query titled "--Which teams most frequently kept clean sheets across all leagues?". The query is as follows:

```
--Which teams most frequently kept clean sheets across all leagues?  
  
SELECT t.team_long_name,  
       SUM(CASE WHEN m.home_team_api_id = t.team_api_id AND m.away_team_goal = 0 THEN 1 ELSE 0 END  
            + CASE WHEN m.away_team_api_id = t.team_api_id AND m.home_team_goal = 0 THEN 1 ELSE 0 END) AS clean_sheets  
FROM soccer_proj."Team" t  
JOIN soccer_proj."Match_tbl" m  
  ON m.home_team_api_id = t.team_api_id OR m.away_team_api_id = t.team_api_id  
GROUP BY t.team_long_name  
ORDER BY clean_sheets DESC  
LIMIT 30;
```

Below the query editor, the results are displayed in a table with columns "team\_long\_name" and "clean\_sheets". The table shows the top 9 teams by clean sheets. Celtic is at the top with 153 clean sheets. A tooltip for the first row shows "ODIN ID: emmanart" and "Name: Emmanuel Arthur".

| team_long_name    | clean_sheets |
|-------------------|--------------|
| Celtic            | 153          |
| FC Barcelona      | 140          |
| Manchester United | 133          |
| Juventus          | 132          |
| FC Porto          | 128          |
| Atlético Madrid   | 125          |
| FC Bayern Munich  | 124          |
| Chelsea           | 123          |
| Ajax              | 122          |

The bottom status bar indicates the current page is 1 of 30, with 200 rows selected.

# Business applications

Why this project matters outside class

Club / league operations: season KPIs (goals, clean sheets, home vs away) for coaching and scouting

Media & broadcasting: compare markets (e.g. avg goals by country) for scheduling and storytelling

Betting & risk (data in Match\_tbl): historical odds vs outcomes; favorite win rates

Talent / HR analog: Player\_Attributes over time  $\approx$  employee skill ratings for hiring and development

Same database patterns as retail (product sales by region) or finance (metrics over time)

# Thank you

## Questions?

Deliverable 4 write-up + video link in submission